Architecture preprint · 19 October 2026 · v0.1.3

DOI: 10.5281/zenodo.23467859

IntentProbe: One Local Model Pass, Many Task-Specific Probes

Public architecture note v0.1.3
Author: Wan Sheng Leung
Publisher: MCPware, a product and research brand of GEASS LABS INC.
Published: 18 October 2026 · **Updated:** 19 October 2026

Activation probes for AI-agent guardrails and classification on CPU. A small local model serves many independent heads from a single forward pass, without generating a verdict for every check.

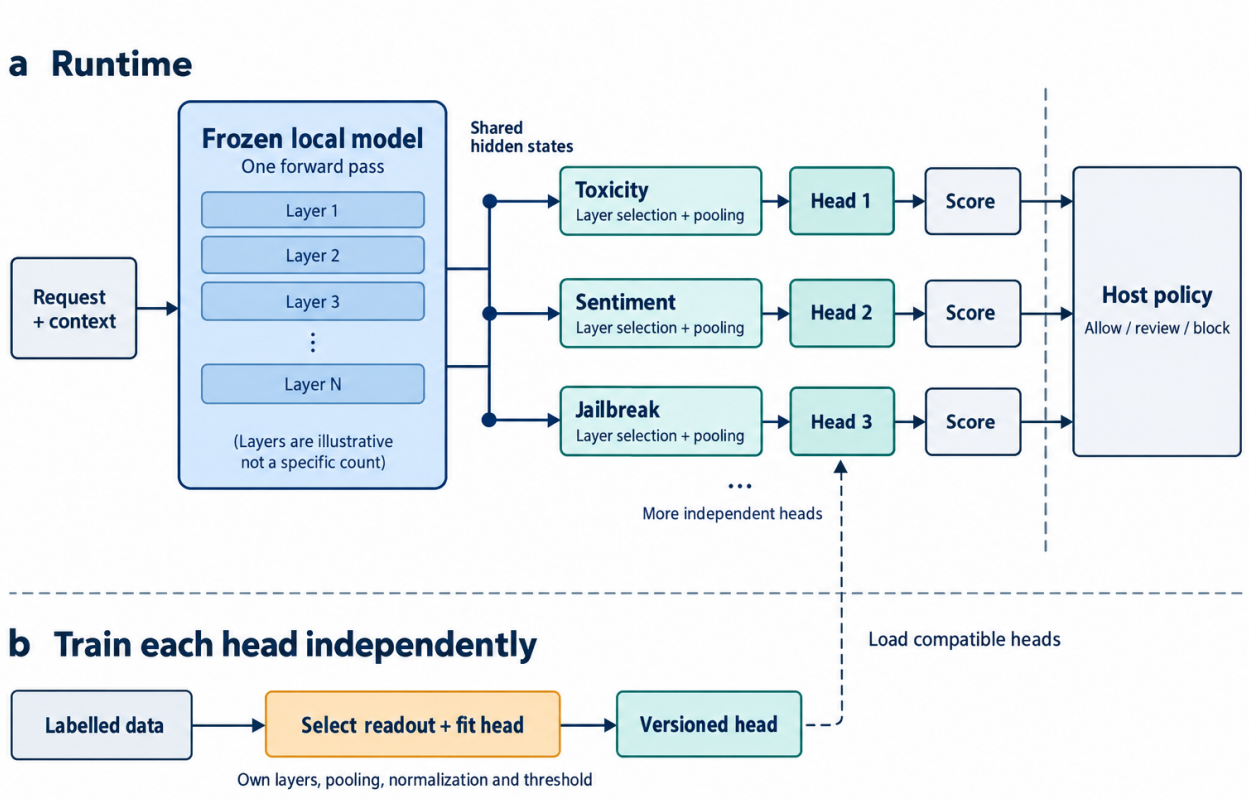


Figure 1. The activation backplane: one model pass that every probe plugs into. (a) At runtime the frozen model reads the request and context once. Its hidden states feed independently trained heads. Each head selects its own layers and pooling and returns a score. (b) Offline, labelled data is used to fit each head and produce a versioned artifact. The shared pass and head application are measured; host policy and compatible-head loading are specified integration points.

Probe models and local scorers · Hugging Face publisher · Reproduce the 16-head benchmark

Measured on one six-core CPU: one Qwen2.5-0.5B forward plus one probe averaged 68.63 ms; the same shared path plus 16 probes averaged 73.34 ms, a 6.9% increase (n=80, paired 95% CI [1.056, 1.078]). Probe work itself rose from 0.38 ms to 1.85 ms.

A language model can carry a useful pattern in its hidden states without turning that pattern into a reliable `YES` or `NO` answer. IntentProbe reads the pattern directly.

Suppose one agent turn needs ten checks: toxicity, sentiment, spam, authorization, and so on. You could ask an LLM each question separately. IntentProbe takes another route: one small local model reads the input, and trained heads turn its hidden states into the different scores.

Beside Claude, Codex, or another hosted agent, that costs one extra local read. All compatible heads share that sidecar pass. If the agent's own model is local and exposes hidden states, heads trained for it can attach to that pass instead.

The short version

Think of the local language model as a brain and each activation head as a small meter attached to it. The model reads an input once. While it reads, its hidden layers react differently to toxicity, sentiment, spam, jailbreak attempts, hostility and other concepts. A separately trained head reads the part of that reaction that matters for one task.

The heads do not ask the model to explain itself. They turn hidden-state patterns into scores using decision boundaries learned from labelled examples.

That gives the runtime a simple shape:

1. render the request and context once;
2. run one frozen local model forward pass;
3. expose the required hidden states through the **activation backplane** -- one model pass that every probe plugs into;
4. apply many small, independently versioned heads;
5. calibrate the scores and pass them to host policy;
6. record what ran in a receipt.

The expensive part is the model forward pass. In the measured CPU runtime, total mean latency was 6.9% higher at K=16 than at K=1. Recorded probe work accounted for 1.47 ms of the 4.70 ms total increase; the remaining 3.23 ms is unattributed. The shared path requests all hidden outputs even at K=1, so this is a backplane baseline rather than an optimized single-probe path.

The terminology is deliberate. The **activation backplane** is the one model pass that every probe plugs into. **Probe** is the public name for a trained task component. **Head** is the technical runtime object attached to the backplane. **Readout** is the recipe that says which token region, layers, pooling and normalization the head uses. **Detector** is reserved for a security product whose job is detection.

The model may know more than it can say

Knowing something and producing the requested answer are different interfaces.

When a direct LLM judge is asked to answer `YES` or `NO`, the model has several jobs. It must understand the prompt, match the writer's label definition, map its internal representation to two output tokens, follow the response format and avoid adding anything else. The wording of the prompt can change the result.

An activation head takes a shorter path. It reads the model's internal response before generation and learns the task boundary from labelled examples. A prompt can describe a rule in words. A trained head can compress thousands of examples of that rule into a small set of weights.

The important point is not that one first-generation head must always be more accurate than every other method. It is that useful task information can already exist inside the model, and labelled examples can teach a small readout how to recover it without asking the model to generate a verdict.

The architecture does not freeze the quality of its first heads. A head can be retrained with better labels, a different layer or pooling recipe, a nonlinear classifier, or a newer local model. The manifest contract is designed so that one compatible head can change without changing the activation interface or retraining unrelated probes. Benchmarking decides whether a particular head is ready for a particular job; it does not decide whether the shared architecture can keep improving.

This gives IntentProbe two different kinds of progress:

- system progress: one forward pass, shared activations, cheap fan-out, versioned receipts, and policy separation; and
- probe progress: better data, models, readout recipes, calibration, and task-specific heads.

The first systems benchmark measures the shared structure. Individual task benchmarks measure the replaceable components installed on top of it.

What the first systems benchmark found

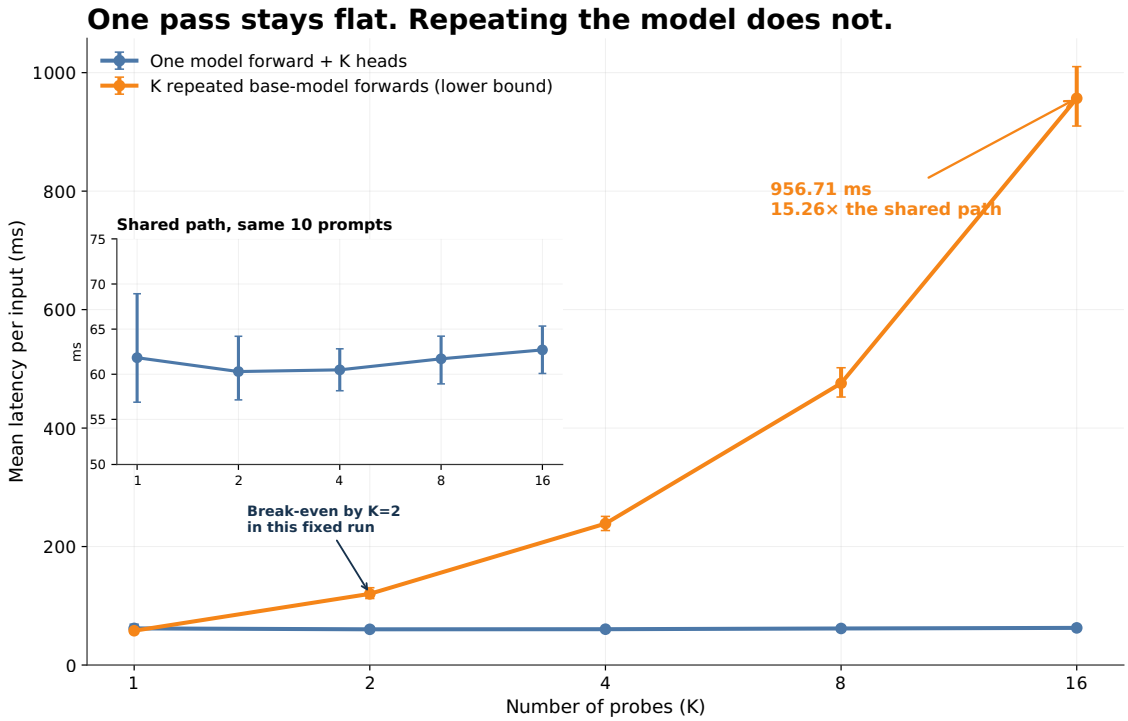
We froze 16 independently trained binary heads on one exact Qwen2.5-0.5B revision. The suite covers toxicity, jailbreak attempts, spam, positive sentiment, five emotion labels and seven conversational pragmatics labels.

The recorded six-core CPU benchmark answers two separate questions with two explicit panels:

Question	Inputs	Measured result
What changes when one forward serves 1 versus 16 heads?	80 held-out texts per K	`68.63 → 73.34 ms`; paired ratio `1.069`, 95% CI `[1.056, 1.078]`
What changes when 16 probes share one pass instead of repeating the model?	Same 10 prompts in both arms	`62.70 vs 956.71 ms`; repeated/shared ratio `15.26`, 95% CI `[15.04, 15.49]`

The repeated-forward measurement is deliberately conservative. It omits the task prompt, language-model output head, label scoring, parsing and generation required by a full LLM judge.

Across the 80-row scaling panel, applying all 16 heads took 1.85 ms per input. Peak process memory at 16 heads was 0.73 MiB above the one-head run, and the saved probe artifacts occupied 386,121 bytes.



Qwen2.5-0.5B · batch 1 · matched 10-prompt panel · error bars are 2,000-resample row-bootstrap 95% CIs. The repeated-forward arm omits task prompts, LM-head scoring and generation.

One common model does not mean one universal layer

The shared backbone is common. The readout recipe is not.

Across the completed model-by-task matrix, the best hidden-state location changed with both task and model. Masked-mean pooling dominated sentiment and toxicity. Spam split evenly between masked mean and masked max. Toxicity and jailbreak detection selected different readouts even when they saw the same input rows from the same model.

Each head therefore carries its own manifest:

- task and label contract;
- exact model and tokenizer revision;
- deterministic input renderer;
- token region and pooling method;
- hidden output or layer window;
- normalization and head family;
- calibration and operating thresholds;
- artifact hashes and benchmark evidence.

The runtime shares the forward pass, not an unsupported assumption that every task lives in the same layer.

Why this can be more practical than LLM-as-a-judge

The architecture becomes most useful when one workflow needs many task-specific scores on the same input.

Instead of writing, tuning and running a separate prompt for every check, one local backbone pass can serve a bank of narrow heads. Each head returns a continuous score and keeps its own threshold. The specified registry is designed to install, update, disable or roll back one compatible head without changing the backbone or unrelated heads.

With the model revision, renderer, readout, weights and threshold frozen, the score is also easier to reproduce and audit than a generated answer. A temperature-zero judge can be deterministic too, but it still depends on prompt wording, chat templates, label tokens and output handling.

This is an architecture and control argument, not a claim that one frozen set of heads is the final answer. Each admitted head still has to pass its own held-out quality, calibration, and shift checks. If it does not, the component can be retrained or replaced while the shared runtime remains the same.

Where it can run

Today, IntentProbe can run as a local sidecar beside a hosted agent such as Claude or Codex. The sidecar model exposes its own hidden states and produces local policy signals without access to the hosted model's internals.

If the primary agent model is local and exposes hidden states, the same architecture can attach its heads directly to that model. The model family is replaceable. Each replacement still needs its own readout search, training and held-out validation because probe weights do not transfer automatically between backbones.

What is established, and what comes next

The current evidence establishes that one frozen 0.5B model can serve 16 real, independently trained heads with nearly flat marginal latency and memory in this CPU environment. It also establishes that readout choice is task-specific. Different tasks can select different layers, pooling rules, and heads while sharing the same model pass.

The implementation boundary is equally specific:

- the PyPI package ships the single-head tool-poisoning path;
- the 16-head shared runtime is measured, reproducible research code; and
- the general manifest registry, hot-loading and multi-head receipt emitter are specified and schema-tested, but are not yet the public PyPI product surface.

The next benchmark adds three comparisons:

- one prompted call that asks the same model for all labels at once;
- separate judge calls with a reusable prompt prefix and caching enabled;
- a measured fleet of compact task-specific classifiers on the same machine and inputs.

Those tests will locate the full accuracy, latency and memory frontier. They do not change the result already measured: once compatible activation heads exist, many of them can share one model forward at very low marginal cost, and each head can improve independently afterward.

Where this fits

There is already useful work on activation-probe guardrails. Goodfire and Rakuten deployed a sidecar for PII detection and described adding more probes at low marginal cost. SingProbe integrates intent, safety, and hallucination monitoring with the model that generates the response. BERTology studies token and layer selection for classification. Lion Warden offers a compact encoder with seven jointly trained security heads and CPU inference.

IntentProbe brings the local deployment, independent task heads, per-head readout recipes, and CPU measurements together in a reproducible architecture. The current runtime measures K=1, 2, 4, 8, and 16 heads. Its public probe catalog also gives readers weights and scorers to try.

Evidence and implementation record

- Research manuscript: IntentProbe: Many Task-Specific Probes from a Single Frozen Small-Model Forward Pass
- Complete implementable architecture disclosure
- One-forward scaling result
- Probe-versus-judge sentinel
- Readout manifest schema
- Multi-head receipt schema

Citation

Leung, Wan Sheng. 2026. IntentProbe: One Local Model Pass, Many Task-Specific Probes. Public architecture note v0.1.3. MCPware, a product and research brand of GEASS LABS INC.

IntentProbe (2026) runs a small frozen local model on CPU as a sidecar and shares one forward pass across independently trained activation probes, each with its own readout manifest.

Download BibTeX with the author handle preserved.